CareerSphere AI: key points

Final-year project · V. Venkata Rupin (2311CS010689), E. Manideep Reddy (2311CS010182), J. Pruthvi Sai (2311CS010264), S. Jaffer Ali (2311CS010639) · Print this and keep it for the review and viva.

1. The project in one minute

What it is. A campus placement platform for students, recruiters and the placement cell. Everything is built on one simple, checkable rule: *how much of a job's required skills a student has*. Because the rule is exact, the system can answer "what if" questions thousands of times a second and explain every number.

The problem it solves. Normal placement portals record what happened. They don't tell a student *which skill to learn*, don't show recruiters *good students who didn't apply*, and don't tell the placement cell *which workshops to run*.

Students

- Career Map: every job placed by match score
- Placement profile: résumé, consent, assessment, video, résumé score
- Skill assessments and certificates (verified levels)
- Opportunity Math, Skill Roadmap, Copilot, leaderboard

Recruiters and placement cell

- Post jobs with skill levels and must-have or nice-to-have skills
- Ranked, explained applicants with verified skills and video
- Hidden Talent: fitting students who didn't apply, with invitations
- Analytics, certificate verification, Workshop Planner

What is new and unique (say these six things)

1. **Opportunity Math**: for each possible skill, it re-scores every job and counts how many come within reach. "Learn Spring Boot: +4 jobs", not just "you're missing Spring Boot".
2. **Skill Roadmap** NEW: exact search over every plan of up to 3 learning steps; it flags skills that only help *together*.
3. **Hidden Talent** NEW: the same rule from the job's side. It shows recruiters consented students who fit but never applied.
4. **Exact Workshop Planner**: greedy selection fails because skills combine, so it checks every combination with bit masks.
5. **Grounded AI**: the Copilot can only read data through 8 tools; the résumé finder keeps a skill only if its quoted line is really in the résumé.
6. **Verified levels**: server-graded tests and placement-cell-checked certificates. Recruiters see the verified level.

1.43 → 4.71
jobs gained per student: best 1 skill vs best 3-step roadmap

35 / 35
students reached by 3 exact workshops (greedy: 32)

10 + 9
hidden fits within reach + one step away

56 tests
39 unit + 17 end-to-end API tests

2. Formulas to know

Skill levels

0 = not known, 1 = Beginner, 2 = Intermediate, 3 = Advanced. A job sets a **minimum level** for each skill and marks it **must-have (weight 2)** or **nice-to-have (weight 1)**.

Match score

> credit κ = 1 if level ≥ needed · ½ if 0 < level < needed · 0 if missing
> score = round(100 × Σ weight × credit ÷ Σ weight)

In code it's done in **half units with integers** so nothing is lost to rounding: `score = (200·half + total) / (2·total)`. With all skills must-have at Beginner, this is just *skills you have ÷ skills required*.

Worked example. Job: Java at Intermediate (must), Spring Boot (must), Git (nice). Student: Java Beginner, Spring Boot, Git.
Half units: Java 2 of 4, Spring Boot 4 of 4, Git 2 of 2 → 8 of 10 → **80%**. Explanation shown: "Java (you: Beginner, needs Intermediate) – half credit."

Within reach

> within reach = score ≥ 70% **and** CGPA ≥ job's cutoff

3 of 4 = 75% ✓, 4 of 5 = 80% ✓, 2 of 3 = 67% ×. The CGPA rule never changes the score; it's reported separately ("good match, but CGPA cutoff 8.0").

Opportunity Math

> gain(action) = number of open jobs that are *not* within reach now but *are* after the action

An action is "learn X" or "level up X to Intermediate/Advanced". Ranked by gain, ties broken by how many jobs ask for the skill.

Skill Roadmap NEW

> effort(plan) = total levels gained · "works together" if gain(plan) > sum of each step's own gain

Checks every plan of 1, 2 and 3 steps (at most one step per skill), about **1,000 plans per student**. Keeps the plan with the most jobs, then the least effort. Also gives the cheapest path to each nearby job, or says "CGPA cutoff – skills can't change this".

Why greedy fails (the key argument)

Student has {Git}. Job needs {Java, Spring Boot, Git}.
Java alone → 2/3 = 67% × · Spring Boot alone → 67% × · **both → 100% ✓**
Each skill alone is worth 0, together they're worth 1. That's **not submodular** (no "diminishing returns"), so the greedy 63% (1 − 1/e) guarantee doesn't hold. Greedy sees zero for each and may never pick them. So we use **exact search**.

Workshop Planner

> Reach goal: maximise (students with ≥ 1 job within reach, then total matches)
> Matches goal: maximise (total student–job matches, then students reached)

A workshop raises attendees to Intermediate. All C(22, k) plans are checked: k = 3 → 1,540 plans in 6.5 ms. Bit masks make each check a few AND/OR/popcount operations. If plans × students × jobs > 3×10^8 it falls back to greedy and says so. Each student is invited only to the workshops they actually need.

Résumé score (out of 100) and readiness points

Rule	Points	Rule	Points
Résumé uploaded	15	Verified certificates (up to 2)	5 each
About you, 150+ chars (50+ = 5)	10	GitHub link	5
Skills listed (up to 5)	4 each	LinkedIn link	5
Skills at Intermediate+ (up to 5)	2 each	Introduction video	5
Verified skills (up to 3)	5 each	CGPA, 10th, 12th filled	5

> readiness points = résumé score + 15 × verified skills + 10 × verified certificates + 5 × jobs within reach

Assessment grading

8 questions (4 basic, 4 advanced), 10 minutes, options shuffled, answers **never sent to the browser**, graded on the server. **8/8 → Advanced · 6–7 → Intermediate · 4–5 → Beginner · below 4 → nothing, retry after 24 h.** 15 skills × 8 = 120 questions.

3. Numbers to remember (demo batch)

Item	Value	Item	Value
Students / jobs / skills	35 / 12 / 22	Student–job pairs	420
Students with a job within reach today	26	Matches (≥ 70%) today	44
Opportunity Math: mean jobs gained	1.43	Most-demanded skill instead	1.03
Students whose suggestion unlocks nothing	7 (vs 20)	Roadmap 2 steps / 3 steps	2.89 / 4.71
Students helped by the roadmap	35 of 35	Multi-step plans marked "works together"	59 of 65
Hidden Talent: within reach / one step	10 / 9	Applications made by consented students	14
3 workshops, exact: students / matches	35 / 138	3 workshops, greedy	32 / 76
Exact plan for k = 3	Spring Boot, Java, Git	Greedy plan for k = 3	Docker, SQL, Git
Planner time, k = 3	6.5 ms	Plans checked, k = 3	1,540
Demo student: jobs within reach	3 of 12	After adding résumé skills	8 of 12
Unit tests / API tests	39 / 17	Backend code	68 files, ~6,400 lines

4. Technology, and what each piece does

Layer	Technology	Why we used it
Frontend	React 18, Vite, React Router, Axios, SVG	Fast single-page app; Career Map drawn in SVG; one Axios client adds the token
Backend	Java 17, Spring Boot 3.3, Spring Web	REST API with layers: controller → service → repository
Security	Spring Security, JWT, BCrypt	Stateless login token (8 h); passwords hashed; each URL prefix open to one role
Data	MySQL 8, JPA/Hibernate; H2 for tests	Tables generated from entities; unique (job, student) stops double applying
AI	Google Gemini, PDFBox, Apache POI	Copilot via function calling; résumé text from PDF/Word; works offline without a key
Apps	Xcode (SwiftUI), Android Studio (Kotlin), VS Code	Mac/iPhone and Android apps wrap the web app; VS Code tasks and debugging
Delivery	JUnit 5, MockMvc, Docker, Railway	Tests; one Docker image; online hosting with HTTPS

How a request travels: click → React → Axios with `Authorization: Bearer <JWT>` → Spring Security checks token and role → controller → service (MatchService etc.) → JPA → MySQL → JSON back → screen updates.

5. Viva questions and short answers

Q1. Why not use machine learning for matching?
We have no large labelled history, and a learned score can't be explained line by line. Our rule is transparent and lets us compute "what if" exactly. ML weights are future work once real outcomes exist.

Q2. Why 70%?
It admits 3 of 4 and 4 of 5 skills (close enough to be shortlisted) but not 2 of 3. It's a design choice, kept in one constant, and the human evaluation is meant to check it.

Q3. What is submodularity and why does it matter?
"Diminishing returns": adding an item helps less the more you have. Greedy is guaranteed 63% of optimal only then. Our problem has a threshold, so two skills can be worthless alone but valuable together. That breaks the guarantee, so we search exactly.

Q4. Isn't exact search too slow?
Not at college scale: 1,540 plans for 3 workshops (6.5 ms), about 1,000 roadmap plans per student. Bit masks make each check a few CPU operations. There's an automatic greedy fallback for huge inputs.

Q5. How is the roadmap different from Opportunity Math?
Opportunity Math looks one step ahead. The roadmap checks pairs and triples, so it finds plans like "Java + SQL → 3 jobs" when each alone gives 0. That helped all 7 students who had no useful single skill.

Q6. How does Hidden Talent respect privacy?
Only students who gave placement consent appear, and only for jobs they haven't applied to. The recruiter just sends an invitation; the student decides.

Q7. How do you stop the AI from making things up?
The Copilot can only read through 8 read-only tools scoped to the logged-in student, and it lists the data it used. The résumé finder drops any skill whose quoted line isn't really in the résumé. Actions need the student's click.

Q8. What happens without an AI key?
Offline mode: rule-based answers from the same tools, and pattern-based résumé reading (JavaScript ≠ Java, "react quickly" ≠ React).

Q9. How is login secured?
BCrypt hashes, a signed JWT valid for 8 hours with user id and role, role rules on every URL prefix, ownership checks in services, and no public admin registration.

Q10. Can a student cheat the assessment?
The answers never reach the browser; option order is stored on the server; time is checked on the server; fewer than 4 correct means a 24-hour wait. The bank is small, so it's for screening, not certification.

Q11. How are uploads made safe?
Size limits (5 MB résumé and certificate, 25 MB video), file type checked from the first bytes, random UUID names, no user-chosen paths.

Q12. How did you test it?
39 unit tests on the rules with hand-checkable cases, 17 API tests through the real security chain on H2, and a browser walk-through of every screen at desktop and phone width. Changing ≥ to > made a test fail, so the tests catch bugs.

Q13. What are the limitations?
Fictional data; fixed weights and threshold; self-reported levels unless verified; workshops ignore capacity and cost; Gemini free tier only for demo data.

Q14. Will it work as a real launched website?
Yes, technically: it deploys with Docker on Railway over HTTPS. For a real launch it would still need real college data, email notifications, backups and a privacy policy.

Q15. What is the ER structure?
Users (with role) → one student or recruiter profile; skills shared; student skills with level and verified level; job skills with min level and weight; applications and invitations unique per (job, student); certificates and assessment attempts per student and skill.

Q16. What would you do next?
Human evaluation, learning weights from real outcomes, capacity-aware workshop planning, notifications, and a real placement-season study.

6. Demo steps (about 10 minutes)

Role	Email	Password
Student	student@careersphere.ai	student123
Recruiter	recruiter@careersphere.ai	recruiter123
Placement cell	admin@careersphere.ai	admin123
Student with invitation	ananya@student.demo	student123

1. **Start:** `./run.sh` in the project folder (or Run in Xcode / the VS Code task "CareerSphere: start everything"). Open `http://localhost:5173`.
2. **Student → Career Map:** "3 of 12 jobs within reach". Click the *Spring Boot* lever: 4 jobs slide inside the ring. Say: "every dot is a job, distance = 100 − score".
3. **Opportunity Math:** Spring Boot first, +4 jobs. **Skill roadmap:** 1 step +4, 2 steps +5, 3 steps +6. Point to "works together" by logging in as `rohan@student.demo`: nothing helps alone, Java + SQL gives +3.
4. **Placement profile:** upload `demo/Aarav_Sharma_Resume.pdf`, accept the 6 suggested skills (each with its quoted line), give consent. Jobs within reach go 3 → 8.
5. **Skill assessment:** start one, show the timer and that grading is on the server.
6. **Copilot:** ask "Which skill should I learn next?" and show the "data used" line.
7. **Apply** to a job. **Recruiter →** My jobs → Java Backend Developer → applicants ranked with reasons; change a status.
8. **Hidden Talent:** Backend Engineer – Payments → 2 within reach, 2 need Docker → *Invite* Ananya. Log in as Ananya → Job listings → invitation banner.
9. **Placement cell:** Verify certificates → verify one. Workshop planner → 3 workshops → **26 → 35 students**, "one at a time would reach 32".
10. **Close:** "Every number you saw can be checked by hand from the skills. That's the point."

If something goes wrong

Problem	Fix
Old data or odd numbers	`mysql -u root -e "DROP DATABASE careersphere"`, then `./run.sh` again (demo data is re-created)
Port 8080 or 5173 in use	`./stop.sh`, then start again
Copilot says offline	No key set; that's fine for the demo. Key goes only in `~/.zshrc` as `GEMINI_API_KEY`, never in the project
MySQL not running	`brew services start mysql`
Android emulator shows nothing	Servers must run on the Mac; the app opens `10.0.2.2:5173` (change it in the app's settings for a real phone)
Run the tests	`cd backend && mvn test`

Files to bring

Project zip · Thesis (Word + PDF) · Research paper (Word + PDF) · Slides · this guide · the sample résumé for the demo.

All data in the system is fictional demo data. Numbers above are from that demo batch.